BAYERO UNIVERSITY, KANO
Faculty of Computing
ITC3304: Web Application Development
By

Engr. Umar Balarabe ubalarabe.cs@buk.edu.ng
Mal Ahmad T. Garba atgarba.it@buk.edu.ng

Course Overview: This course aims to provide students with a comprehensive understanding of PHP programming language and its applications in web development. Students will learn the basics of PHP, server-side scripting, and how to build dynamic, database-driven web applications.

Week 1:

- Introduction to PHP

1. Intended Learning Objectives:

At the end of this Module, the student will be able to understand the following:

1) What PHP is all about and its origin.
2) Key features of PHP.
3) Application of PHP and its workflow.

2. Definition:

PHP (Hypertext Preprocessor) is a widely-used server-side scripting language designed for web development. It was originally created by Rasmus Lerdorf in 1994 and is now maintained by The PHP Group. PHP is an open-source, free-to-use language, and it is one of the most popular choices for building dynamic websites and web applications.

2.1. Key Features of PHP:

- Server-Side Scripting: PHP is primarily used on the server-side, meaning that it runs on the web server and generates HTML output that is sent to the client's web browser. This allows developers to create dynamic web pages and interact with databases and other resources.
- Embedded in HTML: PHP code can be embedded directly into HTML, making it easy to mix dynamic content with static content in the same file. PHP code is enclosed in special delimiters (<?php ... ?>) that tell the server to execute the enclosed code.
- Extensive Database Support: PHP offers built-in support for various databases, including MySQL, PostgreSQL, SQLite, and more. This enables developers to create database-driven applications and easily manage data.
- Platform Independence: PHP is a cross-platform language, which means it can run on various operating systems like Windows, macOS, Linux, etc., making it a versatile choice for web development.
- Large Community and Rich Ecosystem: PHP has a vast community of developers, and there is a wealth of libraries, frameworks, and tools available to simplify development tasks and enhance the functionality of PHP applications.
- Easy to Learn: PHP is known for its relatively simple and straightforward syntax, making it accessible to beginners while still offering powerful capabilities for advanced developers.

2.2. PHP Workflow:

- Client Sends a Request: When a user accesses a PHP-based web application through a web browser, the browser sends a request to the web server.
- Server Executes PHP Code: The web server receives the request and locates the PHP files associated with the requested URL. It then passes the PHP files to the PHP processor (usually a module or a CGI executable).

- PHP Processes the Code: The PHP processor parses the PHP code, executes it, and generates HTML content dynamically based on the code's logic and database interactions.
- Server Sends Response: The PHP-generated HTML content is sent back to the web server, which, in turn, sends it to the user's web browser.
- Client Receives the Response: The user's web browser receives the HTML response and renders the web page for the user to interact with.

2.3. Use Cases for PHP:

- Dynamic Websites: PHP is commonly used to create dynamic websites with changing content based on user interactions or data from databases.
- Web Applications: PHP's ability to handle data and interact with databases makes it well-suited for building web applications, such as content management systems (CMS), e-commerce platforms, and social networks.
- Server-Side Scripting: PHP is used for various server-side tasks, like form processing, data validation, user authentication, and more.
- API Development: PHP can be used to create APIs (Application Programming Interfaces) to enable communication between different web applications and services.
- Command-Line Scripting: PHP can be used to write command-line scripts for performing various tasks on the server.

3. Introduction to web development and server-side scripting

Web development is the process of creating and maintaining websites and web applications that are accessible through the internet. It involves a combination of front-end development and back-end development.

3.1. Front-End Development:

Front-end development is concerned with the user interface and user experience of a website or web application. Front-end developers work on the client-side of the application, which is what the users interact with directly in their web browsers.

3.1.1. Key aspects of front-end development include:

- **HTML** (Hypertext Markup Language): It provides the structure and content of a web page, defining elements like headings, paragraphs, images, links, and more.
- **CSS** (Cascading Style Sheets): CSS is used to control the layout, presentation, and appearance of HTML elements, defining styles such as colors, fonts, spacing, and positioning.
- **JavaScript**: JavaScript is a programming language that allows for dynamic and interactive elements on a web page. It enables developers to create features like animations, form validation, and real-time updates without requiring a page refresh.
- Other front-end languages include but are not limited to : React, Vue, Angular, Django etc.

3.2. Back-End Development:

Back-end development involves server-side programming and focuses on the server's functionality and the application's underlying infrastructure.

3.2.1. Key aspects of back-end development include:

- **Server-Side Scripting**: Server-side scripting languages like PHP, Python, Ruby, and Node.js are used to process user requests, interact with databases, perform calculations, and generate dynamic content that is sent to the user's web browser.

- **Databases**: Back-end developers work with databases like MySQL, PostgreSQL, MongoDB, etc., to store and retrieve data required for the web application.
- **Web Servers:** Web servers like Apache, Nginx, and Microsoft IIS handle incoming HTTP requests and respond with the appropriate resources, such as HTML pages, CSS, JavaScript, or images.
- **Security and Authentication**: Back-end developers implement security measures to protect sensitive data, handle user authentication, and prevent common web vulnerabilities.

3.3. Web Development Workflow:

- **Requirement Gathering**: Understanding the project's requirements, target audience, and desired features.
- **Design**: Creating wireframes and mockups to visualize the website's layout and user interface.
- **Front-End Development**: Building the user interface using HTML, CSS, and JavaScript.
- **Back-End Development:** Developing the server-side logic and connecting it to the database.
- **Testing**: Conducting thorough testing to identify and fix any bugs or issues.
- **Deployment**: Uploading the website or web application to a web server for public access.
- **Maintenance**: Regularly updating and maintaining the website to ensure it runs smoothly and stays secure.

3.4. Introduction to Server-Side Scripting

Server-side scripting is a key component of web development and is responsible for processing user requests on the server and generating dynamic content that is sent back to the client's web

browser. Server-side scripts run on the web server and interact with databases, perform calculations, handle authentication, and execute other server-side tasks.

3.4.1. Advantages of Server-Side Scripting:

- Dynamic Content: Server-side scripting allows websites to display personalized and dynamic content to users based on their interactions or data stored in databases.
- Data Processing: Server-side scripts can handle complex data processing tasks, such as form submissions, data validation, and calculations.
- Security: Critical operations, like database access and user authentication, are handled on the server to enhance security and protect sensitive information.
- Code Protection: Server-side scripts are not visible to users, making it more difficult for potential attackers to study or manipulate the code.
- Platform Independence: Since server-side scripts run on the web server, they are not dependent on the client's operating system or web browser.

3.5. Conclution

Common server-side scripting languages include PHP, Python (with frameworks like Django and Flask), Ruby (with Ruby on Rails), Node.js (with Express.js), and more. The choice of language depends on the project's requirements, the developer's expertise, and the desired performance and scalability of the application.

3.6. Setting up PHP environment (XAMPP, WAMP, or similar)

3.6.1. Installation

XAMPP is a popular cross-platform software package that provides a complete web server solution for local development. It includes Apache, MySQL, PHP, and Perl, making it easy to set

up and run a local web server environment on your computer. Here's a step-by-step guide on how to install XAMPP:

1. Download XAMPP: Go to the official XAMPP website (https://www.apachefriends.org/) and download the appropriate version of XAMPP for your operating system (Windows, macOS, or Linux).
2. Run the Installer: Once the download is complete, run the installer with administrator privileges (on Windows) or as a superuser (on macOS/Linux). This will ensure that XAMPP can install and configure necessary components correctly.
3. Choose Components: During the installation process, you will be prompted to choose which components you want to install. By default, Apache, MySQL, PHP, and phpMyAdmin will be selected. You can leave them as they are unless you have a specific reason to exclude any of them.
4. Select Installation Directory: Choose a directory where you want to install XAMPP. The default location is usually fine for most users, but you can change it if needed.
5. Complete the Installation: Allow the installer to complete the installation process. This may take a few minutes, depending on your system's speed.
6. Start XAMPP Control Panel: Once the installation is finished, launch the XAMPP Control Panel. On Windows, you can find it in the Start Menu or use the shortcut created on your desktop.
7. Start Apache and MySQL: In the XAMPP Control Panel, you'll see a list of available services (Apache, MySQL, etc.). Click the "Start" button next to Apache and MySQL to start these services.
8. Check the Installation: Open your web browser and go to http://localhost/. You should see the XAMPP welcome page, confirming that your local web server is up and running.
9. Testing PHP: To test if PHP is working correctly, create a new file (e.g., test.php) in the "htdocs" folder inside your XAMPP installation directory. Add the following code to the file:

```
<?php
php phpinfo();
?>
```

Save the file and access it through your web browser by going to **http://localhost/test.php.** You should see a PHP information page displaying various details about your PHP configuration.

Accessing phpMyAdmin: To manage your MySQL databases, you can access phpMyAdmin by going to http://localhost/phpmyadmin in your web browser.

That's it! You have successfully installed XAMPP on your computer. You can now use this local server environment for web development and testing purposes. Remember to stop the services from the XAMPP Control Panel when you're not using them to free up system resources.

3.6.2. Folder Location

The folder location is based on the selected path during the installation. However, xampp is installed under drive C, the path will be C:\xampp\htdocs. All the php projects are to be situated under this folder htdocs.

3.6.3. Environment

3.7. Basic PHP syntax and data types

3.7.1. PHP Syntax

PHP is a server-side scripting language designed for web development, and its syntax is quite similar to C and Perl. Here are some essential aspects of PHP syntax:

- Tags: PHP code is usually embedded within HTML using specific tags:

`<?php ... ?>`:

This is the standard opening and closing tag for PHP code.

- Comments: PHP supports two types of comments:

`// Single-line comment`: This is used to comment a single line.

`/* Multi-line comment */`: This is used to comment on multiple lines.

- Variables: In PHP, variables are declared using the dollar sign $ followed by the variable name:

```php
$name = "John";
$age = 25;
```

- Data Types: PHP supports several data types, including integers, strings, floats, booleans, arrays, objects, and more.
- String Concatenation: Strings can be concatenated using the . operator:

```php
$firstName = "John";
$lastName = "Doe";
$fullName = $firstName . " " . $lastName; // Result: "John Doe"
```

3.7.2. Variables

In PHP, variables are used to store and manipulate data. A variable is a symbolic name that represents a value. PHP variables have the following characteristics:

- Variable Declaration: Variables in PHP are declared using the dollar sign $ followed by the variable name. The variable name must start with a letter or underscore and can be followed by letters, numbers, or underscores. Variable names are case-sensitive.

```php
$name; // Variable declaration
$age;  // Another variable declaration
```

- Variable Assignment: After declaring a variable, you can assign a value to it using the assignment operator (=).

```php
$name = "John"; // Assigning a string value to the variable $name
$age = 25;      // Assigning an integer value to the variable $age
```

- Dynamic Typing: PHP is a dynamically typed language, meaning you don't need to specify the data type of a variable explicitly. The data type is determined based on the value assigned to the variable.

```php
$variable1 = 42;         // Integer type
$variable2 = "Hello";    // String type
$variable3 = true;       // Boolean type
$variable4 = 3.14;       // Float type
$variable5 = null;       // NULL type
$variable6 = array(1, 2); // Array type
// ... and so on
```

- Variable Interpolation: When a variable is used within double-quoted strings, its value will be automatically replaced in the string. This is known as variable interpolation.

```php
$name = "John";
echo "Hello, $name!"; // Output: Hello, John!
```

- Global and Local Scope: Variables in PHP can have different scopes. Variables declared outside of a function have a global scope and can be accessed from anywhere in the script. Variables declared inside a function have a local scope and can only be accessed within that function unless explicitly specified otherwise.

- Global Scope: Variables declared outside of any function or class have a global scope. These variables are accessible from anywhere in the PHP script, including functions, classes, and even included files.

Example

```php
$globalVar = 10;

function myFunction() {
    echo $globalVar; // Accessible inside the function
}

myFunction();
echo $globalVar; // Accessible outside the function
```

- Local Scope: Variables declared inside a function have local scope. They are only accessible within that specific function and not outside of it.

Example

```php
function myFunction() {
    $localVar = 5;
    echo $localVar; // Accessible inside the function
}

myFunction();
echo $localVar; // This will cause an error - $localVar is not accessible h
```

Note: If a variable with the same name as a global variable is declared inside a function, it will create a new local variable that shadows the global variable within that function.

Example of shadowing a global variable with a local variable:

```php
$myVar = 100; // Global variable

function myFunction() {
    $myVar = 50; // Local variable (shadows the global variable)
    echo $myVar; // Output: 50 (accessing the local variable)
}

myFunction();
echo $myVar; // Output: 100 (accessing the global variable)
```

- Using Global Variables Inside Functions: To access a global variable inside a function, you can use the global keyword to import the global variable into the function's local scope.

  Example of accessing a global variable inside a function:

```php
$globalVar = 10;

function myFunction() {
    global $globalVar;
    echo $globalVar; // Accessible inside the function (value is 10)
}

myFunction();
```

- Super Global Variables: PHP also has super global variables, which are predefined variables accessible from any scope. Examples of super globals include $_GET, $_POST, $_SESSION, $_COOKIE, $_SERVER, etc. These variables store information related to HTTP requests, server configuration, session data, and more.

  Example of using a super global variable:

```php
// Assuming the URL is: http://example.com/index.php?name=John
echo $_GET['name']; // Output: John
```

- Static Variables: You can create static variables inside functions. A static variable retains its value between multiple calls to the same function. When a variable is declared as static inside a function, it remains in memory even after the function finishes executing. The variable's value persists across subsequent calls to the same function.

  To declare a static variable inside a function, you use the `static` keyword before the variable name. Here's the syntax:

```php
function myFunction() {
    static $staticVar = initialValue;
    // Function code using $staticVar
}
```

Here are some key points to understand about static variables in PHP:

- Initialization: A static variable is initialized only once, during the first call to the function. Subsequent calls to the function do not reinitialize the static variable.
- Persistent value: The value of a static variable persists between different function calls. It means the variable retains its last assigned value across multiple invocations of the same function.
- Function scope: A static variable is still limited to the scope of the function in which it is declared. It cannot be accessed or modified from outside the function.

Example of using a static variable:

```php
function incrementCounter() {
    static $counter = 0;
    $counter++;
    echo "Counter: " . $counter . "<br>";
}

incrementCounter(); // Output: Counter: 1
incrementCounter(); // Output: Counter: 2
incrementCounter(); // Output: Counter: 3
```

In this example, the static variable **$counter** is initialized only once during the first call to **incrementCounter**(), and its value persists across subsequent calls. As a result, each time the function is called, the value of **$counter** increases by 1.

Static variables are useful when you want to maintain state information between function calls without resorting to using global variables. They are often used in scenarios where you need to track how many times a function has been called or keep track of some internal state within the function.

3.7.3. Constants

In PHP, a constant is a variable whose value cannot be changed or modified during the script's execution. Once defined, its value remains constant throughout the entire script. Constants are typically used to store values that should not be altered, such as configuration settings, fixed values, or important identifiers.

Here are the key characteristics of PHP constants:

- Naming convention: Constants are typically written in uppercase letters to distinguish them from regular variables. However, PHP is case-sensitive, so you can use any case for constants, but using uppercase is a common convention.
- Defining a constant: You can define a constant using the define() function or the const keyword (available since PHP 5.3).

  Using `define()` function:

```php
define("CONSTANT_NAME", value);
```

Using `const` keyword:

```php
const CONSTANT_NAME = value;
```

Accessing constants: Once defined, you can access constants using their names (without a dollar sign) anywhere in the script, just like regular variables.

Example of defining and accessing constants:

```php
define("PI", 3.14);
const WEBSITE_NAME = "MyWebsite";

echo PI; // Output: 3.14
echo WEBSITE_NAME; // Output: MyWebsite
```

- **Class constants**: Starting from PHP 5.3, you can also define constants inside classes using the const keyword. These constants are associated with the class and can be accessed using the ClassName::CONSTANT_NAME notation.

  Example of defining and accessing a class constant:

```php
define("PI", 3.14);
const WEBSITE_NAME = "MyWebsite";

echo PI; // Output: 3.14
echo WEBSITE_NAME; // Output: MyWebsite
```

3.7.4. PHP Data Types

PHP supports several data types that allow you to store and manipulate different kinds of data. Here are the main data types in PHP:

- Integer: Represents whole numbers without decimal points. For example:

```php
$age = 25;
$count = -10;
```

- Float (Floating-Point Number): Represents numbers with decimal points. For example:

```php
$price = 9.99;
$pi = 3.14159;
```

- String: Represents a sequence of characters enclosed within single quotes (') or double quotes ("). For example:

```php
$name = "John Doe";
$message = 'Hello, World!';
```

- Boolean: Represents a true or false value. It can be either true or false (without quotes):

```php
$is_active = true;
$is_student = false;
```

- Array: Represents an ordered collection of elements. Arrays can store multiple values, including other arrays. For example:

```php
$numbers = array(1, 2, 3, 4, 5);
$fruits = ['apple', 'banana', 'orange'];
$person = ['name' => 'John', 'age' => 30, 'is_student' => false];
```

- Object: Represents an instance of a class. Objects are used in object-oriented programming. For example:

```php
class Person {
    public $name;
    public $age;
}

$person = new Person();
$person->name = "John";
$person->age = 25;
```

- NULL: Represents a variable with no value. It is used to indicate that a variable has not been assigned a value or to unset a variable. For example:

```php
$variable = NULL;
```

- Resource: Represents an external resource, such as a database connection or a file handle. Resources are typically created and managed by functions in PHP extensions.

3.8. Data manipulation

Run examples on how to manipulate data in PHP

3.9. Operations in PHP

In PHP, operations refer to various types of computations or manipulations that can be performed on variables or data. PHP supports a wide range of operations, including arithmetic operations, assignment operations, comparison operations, logical operations, and more. Let's explore some of the most common operations in PHP:

1. Arithmetic Operations:
    - Addition: $result = $num1 + $num2;
    - Subtraction: $result = $num1 - $num2;
    - Multiplication: $result = $num1 * $num2;
    - Division: $result = $num1 / $num2;
    - Modulus (Remainder): $result = $num1 % $num2;
2. Assignment Operations
    - Assignment: $variable = $value;
    - Addition Assignment: $num += 5; (Equivalent to $num = $num + 5;)
    - Subtraction Assignment: $num -= 2; (Equivalent to $num = $num - 2;)
    - Multiplication Assignment: $num *= 3; (Equivalent to $num = $num * 3;)

- Division Assignment: $num /= 2; (Equivalent to $num = $num / 2;)
- Modulus Assignment: $num %= 4; (Equivalent to $num = $num % 4;)

3. Comparison Operations:
   - Equal to: $result = $num1 == $num2;
   - Not equal to: $result = $num1 != $num2;
   - Identical (equal value and type): $result = $num1 === $num2;
   - Not Identical (not equal value or type): $result = $num1 !== $num2;
   - Greater than: $result = $num1 > $num2;
   - Less than: $result = $num1 < $num2;
   - Greater than or equal to: $result = $num1 >= $num2;
   - Less than or equal to: $result = $num1 <= $num2;
4. Logical Operation
   - AND (&&): $result = $condition1 && $condition2;
   - OR (||): $result = $condition1 || $condition2;
   - NOT (!): $result = !$condition;
5. String Concatenation
   - Concatenation: `$result = $string1 . $string2;`
6. Incrementation and Decrementation
   - Increment: $num++; or ++$num;
   - Decrement: $num--; or --$num;
7. Ternary Operation
   - Shorthand for simple if-else statements: $result = $condition ? $valueIfTrue : $valueIfFalse;
8. Array Operation
   - Accessing elements: $element = $array[index];
   - Adding elements: $array[] = $newValue; or $array[key] = $newValue;
   - Removing elements: unset($array[index]);